

实验报告

邓铸城 -25020007013

August 24, 2026

1 Result

1.1 Q01 含空格文件名的批量整理

1.1.1 实验任务

创建多级目录与带空格名称的文件；输出目录绝对路径，长格式列出包含隐藏文件；复制 txt 文件并保留目录结构，正确处理文件名空格；修改目录、文件权限，生成记录路径与字节数的 inventory.txt。

1.1.2 关键命令

```
1 cd ~/work/q01
2 mkdir -p input/docs input/tmp
3 echo -e "alpha\nbeta" > "input/docs/notes one.txt"
4 echo "hidden" > input/docs/.secret.txt
5 touch input/tmp/empty.txt
6 echo -e "log line1\nlog line2" > input/run.log
7
8 pwd
9 ls -la input
```

1.1.3 现象与说明

文件名包含空格时，必须使用引号包裹文件名，防止 Shell 将空格识别为参数分隔符。ls -la 可以同时展示普通文件和点开头的隐藏文件。cp --parents 在复制时保留原有的目录层级。chmod 用于修改目录与普通文件的访问权限。

```
fernandes8@ubuntu:~/q01$ realpath .  
/home/fernandes8/q01  
fernandes8@ubuntu:~/q01$ ls -la input  
总用量 20  
drwxr-xr-x 4 fernandes8 fernandes8 4096 8月 24 00:05 .  
drwxr-xr-x 5 fernandes8 fernandes8 4096 8月 24 00:10 ..  
drwxr-xr-x 2 fernandes8 fernandes8 4096 8月 24 00:05 docs  
-rw-r--r-- 1 fernandes8 fernandes8 22 8月 24 00:05 run.log  
drwxr-xr-x 2 fernandes8 fernandes8 4096 8月 24 00:05 tmp  
fernandes8@ubuntu:~/q01$ cat inventory.txt  
input/tmp/empty.txt 0  
input/docs/notes one.txt 11  
input/docs/.secret.txt 7
```

图 1: Q01 终端运行截图

1.2 Q02 随机访问日志统计

1.2.1 实验任务

编写 `analyze.sh` 解析 CSV 访问日志。文件不存在时将错误输出到 `stderr`；统计 5xx 状态码出现次数最多的前 2 个 path；计算平均 `latency_ms`，保留两位小数。

1.2.2 核心代码片段

```
1 #!/bin/bash  
2 CSV="$1"  
3 if [ ! -f "$CSV" ]; then  
4     echo "error: file $CSV not exists" >&2  
5     exit 1  
6 fi
```

使用 `>&2` 将错误消息重定向至标准错误输出。结合 `awk`、`sort`、`uniq` 完成日志统计。

```

fernandes8@ubuntu:~/q02$ realpath .
/home/fernandes8/q02
fernandes8@ubuntu:~/q02$ ls -la
总用量 16
drwxr-xr-x  2 fernandes8 fernandes8 4096 8月 24 00:18 .
drwxr-xr-x 31 fernandes8 fernandes8 4096 8月 24 23:51 ..
-rw-r--r--  1 fernandes8 fernandes8 299 8月 24 00:13 access.csv
-rwxr-xr-x  1 fernandes8 fernandes8 515 8月 24 00:18 analyze.sh
fernandes8@ubuntu:~/q02$ cat access.csv
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
fernandes8@ubuntu:~/q02$ ./analyze.sh access.csv
    3 /api/orders
    2 /api/users
avg latency: 193.75
fernandes8@ubuntu:~/q02$

```

图 2: Q02 终端运行截图

1.3 Q03 制造并解决 Git 合并冲突

1.3.1 实验任务

初始化 Git 仓库，在 main 分支创建 config.txt；创建两个 feature 分支修改文件同一行内容；合并产生冲突；解决冲突时保留 mode=safe，新增一行 note=reviewed；查看图形化提交历史。

1.3.2 关键 Git 命令

```

1 cd ~/work/q03
2 git init
3 echo "mode=normal" > config.txt
4 git add config.txt
5 git commit -m "init config mode normal"
6
7 git checkout -b feature-a
8 sed -i 's/mode=normal/mode=safe/' config.txt
9 git commit -am "set mode safe"
10
11 git checkout main
12 git checkout -b feature-b
13 sed -i 's/mode=normal/mode=fast/' config.txt
14 git commit -am "set mode fast"

```

```

15
16 git checkout main
17 git merge feature-a
18 git merge feature-b
19 # 手动编辑冲突文件
20 git add config.txt
21 git commit -m "resolve conflict keep safe add note reviewed"
22
23 git log --all --graph --decorate --oneline

```

冲突解决完成后 config.txt 内容:

```

mode=safe
note=reviewed

```

Git 使用 <<<<<<、=====、>>>>>> 标记冲突区域, 人工编辑选择内容, 再执行 add 与 commit 完成冲突解决。

```

fernandes8@ubuntu:~/q03$ realpath .
/home/fernandes8/q03
fernandes8@ubuntu:~/q03$ cat config.txt
mode=safe
note=reviewed
fernandes8@ubuntu:~/q03$ git status
位于分支 master
无文件要提交, 干净的工作区
fernandes8@ubuntu:~/q03$ git log --all --graph --decorate --oneline
* aa26c91 (HEAD -> master) merge feature-b, resolve conflict keep safe + reviewed note
|
| * ce231ab (feature-b) feature-b set fast mode
| * | df8e311 (feature-a) feature-a set safe mode
|/
* 53d6dba init config mode=normal
fernandes8@ubuntu:~/q03$

```

图 3: Q03 Git 图形化日志截图

1.4 Q04 LaTeX 文档构建, 公式与表格交叉引用

1.4.1 实验任务

补全 LaTeX 源文件, 添加带编号与 label 的公式、带 caption 和 label 的 2 行 2 列表格; 在正文内完成交叉引用, 编译生成 PDF, 保证不出现??。

见公式 (1):

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (1)$$

见表 1:

表 1: 测试数据表

项目	数值
10	20
30	40

再次引用公式 (1)以及表 1。

编译命令：

```
1 latexmk -xelatex report.tex
```

Tool Report

Deng Zhucheng-25020007013

August 24, 2026

1 Result

See equation (1):

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (1)$$

See Table 1, it is a 22 table:

Table 1: Test data table

A	B
10	20
30	40

We refer again to equation (1) and Table 1.

图 4: Q04 生成 PDF 截图

2 课后 Shell 实验

脚本文件 `hw04_quote.sh`、`hw05_redirect.sh`、`hw06_mkdir.sh`、`hw08_check.sh`、`hw09_exec.sh`、`hw11_backup.sh` 存放于仓库 `after-class/` 目录，已经提交到 GitHub。hw02、hw13、hw14、hw17 为纯命令行实验，不上传脚本，仅在报告中附上运行截图。

2.1 hw02 ls 命令参数实验

使用 `ls -n`、`ls -l`、`ls -h`，分别查看文件 UID/GID、详细权限属性、人类可读格式文件大小。

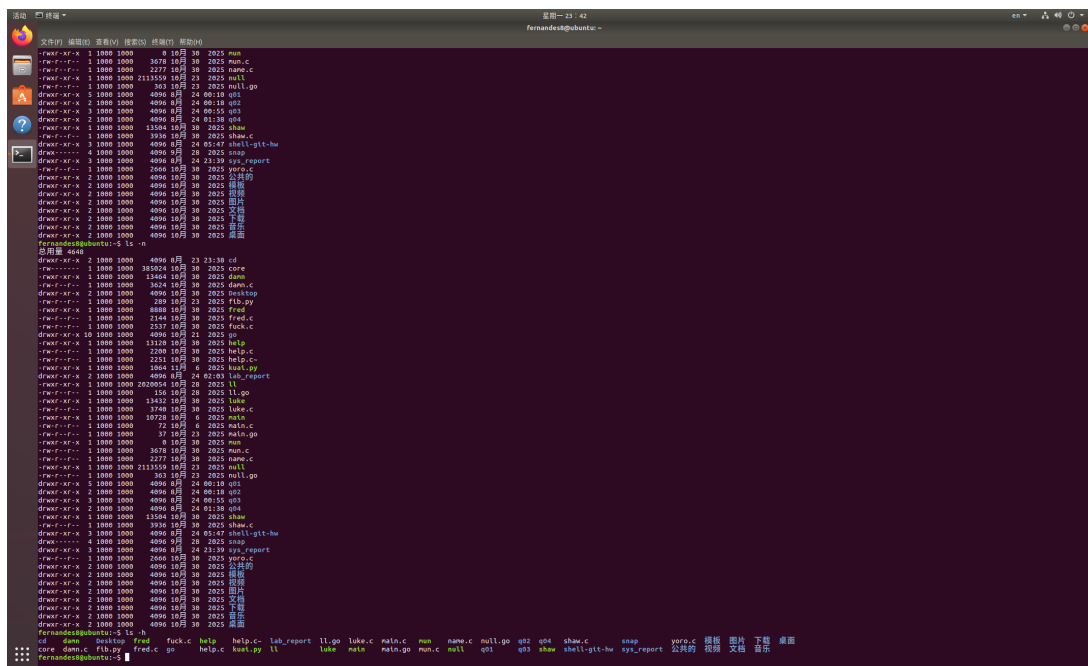


图 5: hw02 ls 参数运行截图

2.2 hw04 单引号与双引号区别

- 单引号 ' ' : 原样输出字符串, 不会解析内部变量。- 双引号 " " : 保留字符串内部空格, 会解析 \$ 开头的变量。

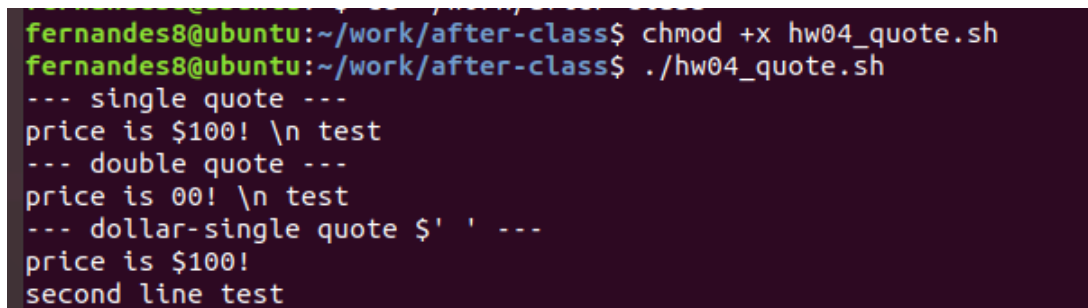


图 6: hw04 引号对比截图

2.3 hw05 stdout 与 stderr 输出重定向

- > out.txt: 重定向标准输出 stdout。
- 2> err.txt: 重定向标准错误 stderr。
- > all.txt 2>&1: 将 stderr 合并到 stdout, 一并写入文件。

```

fernandes8@ubuntu:~/work/after-class$ chmod +x hw05_redirect.sh
fernandes8@ubuntu:~/work/after-class$ ./hw05_redirect.sh
fernandes8@ubuntu:~/work/after-class$ cat out.txt
/tmp:
config-err-lsVCal
ssh-Q8Q9jjwYGTsv
systemd-private-f7ce866e5ef84de8840581c9ae25de34-bolt.service-k69yhw
systemd-private-f7ce866e5ef84de8840581c9ae25de34-colord.service-VhYpXD
systemd-private-f7ce866e5ef84de8840581c9ae25de34-fwupd.service-ovwX1f
systemd-private-f7ce866e5ef84de8840581c9ae25de34-ModemManager.service-VCUwFR
systemd-private-f7ce866e5ef84de8840581c9ae25de34-rtkit-daemon.service-KE90d0
systemd-private-f7ce866e5ef84de8840581c9ae25de34-systemd-resolved.service-EWTw6H
systemd-private-f7ce866e5ef84de8840581c9ae25de34-systemd-timesyncd.service-NxXIXz
VMwareDnD
vmware-root_876-2689209388
fernandes8@ubuntu:~/work/after-class$ cat err.txt
ls: 无法访问'/nonexistent_folder': 没有那个文件或目录
fernandes8@ubuntu:~/work/after-class$ cat all.txt
ls: 无法访问'/nonexistent_folder': 没有那个文件或目录
/tmp:
config-err-lsVCal
ssh-Q8Q9jjwYGTsv
systemd-private-f7ce866e5ef84de8840581c9ae25de34-bolt.service-k69yhw
systemd-private-f7ce866e5ef84de8840581c9ae25de34-colord.service-VhYpXD
systemd-private-f7ce866e5ef84de8840581c9ae25de34-fwupd.service-ovwX1f
systemd-private-f7ce866e5ef84de8840581c9ae25de34-ModemManager.service-VCUwFR
systemd-private-f7ce866e5ef84de8840581c9ae25de34-rtkit-daemon.service-KE90d0
systemd-private-f7ce866e5ef84de8840581c9ae25de34-systemd-resolved.service-EWTw6H
systemd-private-f7ce866e5ef84de8840581c9ae25de34-systemd-timesyncd.service-NxXIXz
VMwareDnD
vmware-root_876-2689209388

```

图 7: hw05 输出重定向截图

2.4 hw06 条件创建目录

脚本判断目标目录是否存在，仅当目录不存在时执行创建操作。

```

fernandes8@ubuntu:~/work/after-class$ chmod +x hw06_mkdir.sh
fernandes8@ubuntu:~/work/after-class$ ./hw06_mkdir.sh
fernandes8@ubuntu:~/work/after-class$ ls -ld /tmp/mydir
drwxr-xr-x 2 fernandes8 fernandes8 4096 8月 24 23:57 /tmp/mydir

```

图 8: hw06 条件创建目录截图

2.5 hw08 判断普通文件是否存在

使用 Shell 条件 [-f filename] 判断文件是否存在，根据结果输出不同提示信息。

```

fernandes8@ubuntu:~/work/after-class$ chmod +x hw08_check.sh
fernandes8@ubuntu:~/work/after-class$ ./hw08_check.sh /etc/passwd
File /etc/passwd exists.
fernandes8@ubuntu:~/work/after-class$ ./hw08_check.sh not_exist_file
File not_exist_file does NOT exist.

```

图 9: hw08 文件存在检测截图

2.6 hw09 Shell 脚本执行权限实验

- 新建 sh 脚本默认没有 x 执行权限，直接 ./test.sh 会报 Permission denied。

- `bash test.sh` 直接调用解释器读取文本，不校验文件 `x` 权限，依然可以运行。
- `chmod +x` 添加执行位之后，才可以使用 `./test.sh` 直接运行脚本。

```
fernandes8@ubuntu:~/work/after-class$ touch test.sh
fernandes8@ubuntu:~/work/after-class$ echo 'echo hello' > test.sh
fernandes8@ubuntu:~/work/after-class$ ls -l test.sh
-rw-r--r-- 1 fernandes8 fernandes8 11 8月 25 00:03 test.sh
fernandes8@ubuntu:~/work/after-class$ ./test.sh
bash: ./test.sh: 权限不够
fernandes8@ubuntu:~/work/after-class$ bash test.sh
hello
fernandes8@ubuntu:~/work/after-class$ chmod +x test.sh
fernandes8@ubuntu:~/work/after-class$ ls -l test.sh
-rwxr-xr-x 1 fernandes8 fernandes8 11 8月 25 00:03 test.sh
fernandes8@ubuntu:~/work/after-class$ ./test.sh
hello
```

图 10: hw09 脚本执行权限截图

2.7 hw11 使用时间戳做文件备份

调用 `date` 命令获取系统当前时间，拼接至备份文件名，每次备份生成不同文件名，避免覆盖旧备份文件。

```
fernandes8@ubuntu:~/work/after-class$ echo "test noet" > notes.txt
fernandes8@ubuntu:~/work/after-class$ chmod +x hw11_backup.sh
fernandes8@ubuntu:~/work/after-class$ ./hw11_backup.sh
fernandes8@ubuntu:~/work/after-class$ ls
all.txt err.txt hw04_quote.sh hw05_redirect.sh hw06_mkdir.sh hw08_check.sh hw11_backup.sh notes-20260825.txt notes.txt out.txt test.sh
```

图 11: hw11 时间戳备份截图

2.8 hw13 统计家目录下文件后缀排行

通过管道组合 `find` 遍历全部文件，`sed` 截取后缀，`sort`、`uniq-c` 统计计数，`sort-nr` 降序排序，`head` 取数量前 5 的后缀。

```
fernandes8@ubuntu:~/work/after-class$ find ~ -type f | sed -E 's/.*\.[^.]*/\1/' | sort | uniq -c | sort -nr | head -5
10635 go
1380 txt
625 s
593 png
407 crt
```

图 12: hw13 文件后缀统计截图

2.9 hw14 `find-print0` 与 `xargs-0` 安全处理文件名

`-print0` 与 `xargs-0` 成对使用，以空字符分隔文件名，安全处理包含空格、特殊字符的文件名；`wc-l` 统计每个 `sh` 脚本代码行数。

```

fernandes8@ubuntu:~/work$ cd ~/work
fernandes8@ubuntu:~/work$ find . -name "*.sh" -print0 | xargs -0 wc -l
 17 ./q02/analyze.sh
   1 ./after-class/test.sh
   9 ./after-class/hw04_quote.sh
   2 ./after-class/hw06_mkdir.sh
   3 ./after-class/hw05_redirect.sh
   6 ./after-class/hw08_check.sh
   2 ./after-class/hw11_backup.sh
 40 总用量

```

图 13: hw14 print0 与 xargs-0 截图

2.10 hw17 awk 条件筛选并交换列

awk 使用条件表达式筛选第一列大于 100 的行，通过 `print $2,$1` 输出实现两列交换。

```

fernandes8@ubuntu:~$ printf "50 x\n150 y\n200 z\n" | awk '$1>100 {print $2,$1}'
y 150
z 200

```

图 14: hw17 awk 筛选交换列截图

3 附录 Git 提交记录

在 /work 目录执行 `git log --oneline` 得到仓库简短提交历史。

```

08916dc (HEAD -> master, origin/master, origin/HEAD) hw09 completed
f3b1112 homework:hw11
5e7ebc7 homework hw08
b18da54 homework:hw06
aed3e1a homework:add hw05
a904481 homework:add hw04
bebdeff q04 completed
4c93075 q03 completed
f610d7a q02 completed
1c04e80 q01 completed

```

图 15: Git 提交日志截图

4 实验总结

1. Shell 处理带空格文件名必须加引号；重定向需要区分标准输出 `stdout` 与标准错误 `stderr`；脚本的 `x` 执行权限和 `bash` 直接读取运行是两套不同机制。
2. Git 两个分支修改同一处内容会触发合并冲突，冲突需要人工编辑文件后再提交。
3. LaTeX 需要多次编译，公式、表格的交叉引用才会生效，防止出现?? 未解析标记。
4. 通过 `find`、`sed`、`awk`、`sort`、`xargs` 管道组合，可以在不写高级程序的情况下完成复杂文本处理任务。